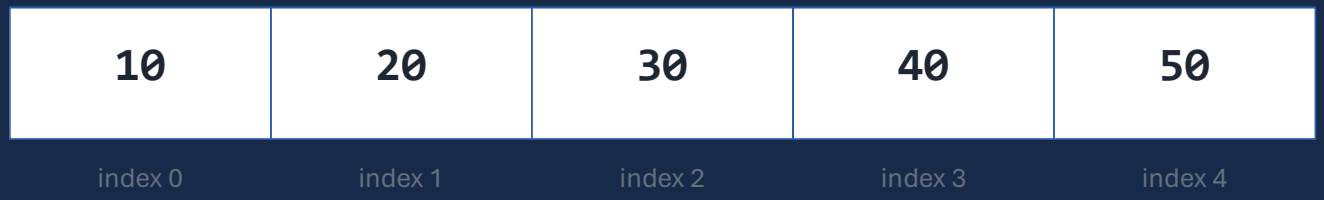
ARRAYS IN C

Introduction • One-Dimensional Arrays • Declaration • Initialization • Reading & Writing



Learning Objectives

By the end of this lesson, students should be able to:

- Explain why arrays are needed in C
- Declare and initialize one-dimensional arrays
- Understand array indexing and memory layout
- Read values into an array using loops
- Write/display array elements using loops
- Develop simple programs using arrays

What is an Array?

An array stores multiple values of the same data type under one name.

Definition

An array is a collection of elements of the same data type stored in contiguous memory locations.



Key idea

Instead of creating many separate variables, we use one array name and access elements using an index.

Why Do We Need Arrays?

Compare individual variables with a single array.

```
int m1, m2, m3, m4, m5;

m1 = 78;
m2 = 84;
m3 = 69;
m4 = 91;
m5 = 76;
```

```
int marks[5];

marks[0] = 78;
marks[1] = 84;
marks[2] = 69;
marks[3] = 91;
marks[4] = 76;
```

Advantage

Arrays make repetitive data storage and processing much easier.

Real-World Examples of Arrays

Student Marks

Store marks of 60 students in one array.

Temperatures

Store daily temperatures for a month.

Prices

Store prices of products in a shop.

Sensor Data

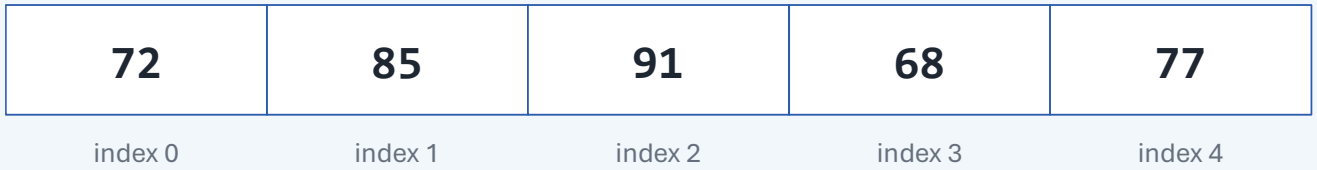
Store repeated readings from a sensor.

Scores

Store scores of players in a game.

Attendance

Store attendance percentages for students.



Types of Arrays

Arrays can have one or more dimensions.

1-D Array

A simple list of values.

Example:
int marks[5];

Access with one index:
marks[2]

2-D Array

Data arranged in rows and columns.

Example:
int a[3][4];

Access with two indices:
a[1][2]

Multi-Dimensional

Arrays with three or more dimensions are possible, but memory and processing become more complex.

Focus of this lesson

We will concentrate on one-dimensional arrays.

One-Dimensional Array

A 1-D array can be visualized as a row of elements.



- All elements have the same data type.
- Each element is identified by an index.
- Elements are stored in contiguous memory.
- The first index is always 0 in C.

Array Declaration

Syntax and meaning of a one-dimensional array declaration.

```
data_type array_name[size];

Examples:
int marks[5];
float temp[7];
char grade[10];
```

data_type

Type of every element: int, float, char, etc.

array_name

The identifier used to refer to the array.

size

Maximum number of elements allocated for the array.

Understanding Array Size

- If size is 5, valid indices are 0 through 4.
- Number of elements = declared size.
- Index is not the same as position when counting from 1.
- Accessing an invalid index can cause undefined behavior.



Valid indices

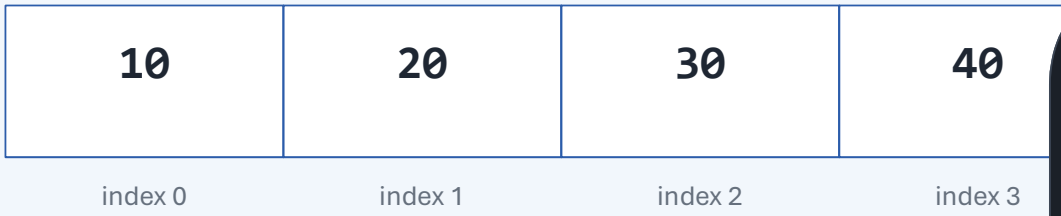
0, 1, 2, 3, 4

Invalid examples

5, 6, -1

Array Indexing in C

The index tells C which element you want.




```c
int a[5] = {10,20,30,40,50};

printf("%d", a[0]);  // 10
printf("%d", a[3]);  // 40
printf("%d", a[4]);  // 50
```

Remember

First element → a[0]
Last element → a[size-1]

Array Initialization

Values can be supplied when the array is declared.

```
int a[5] = {10, 20, 30, 40, 50};

int b[] = {5, 10, 15};

char grade[4] = {'A','B','C','D'};
```

- Initializer values are placed from index 0 onward.
- When size is omitted, C determines it from the initializer list.
- All values must be compatible with the declared type.

Partial Initialization

What happens when fewer values are supplied?

```
int a[5] = {10, 20};

// a[0] = 10
// a[1] = 20
// a[2] = 0
// a[3] = 0
// a[4] = 0
```



Important

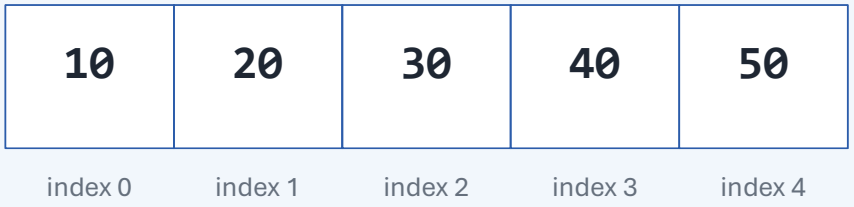
Remaining elements are initialized to zero for this declaration.

Initializing an Array with a Loop

Useful when values follow a pattern or are computed.

```
int a[5];

for (int i = 0; i < 5; i++)
{
    a[i] = (i + 1) * 10;
}
```

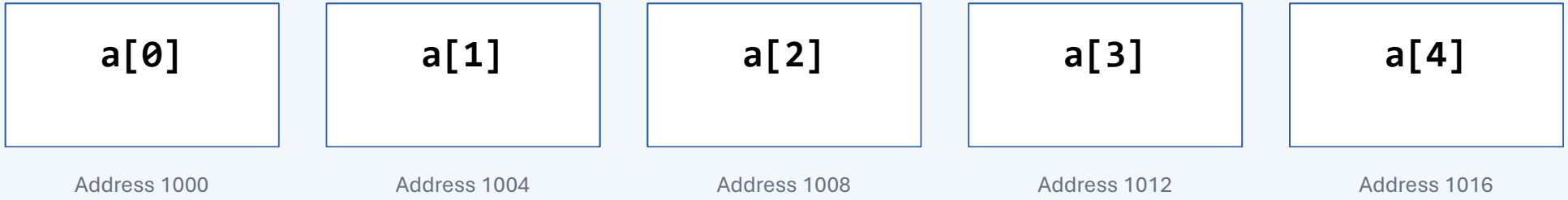


Pattern

i = 0 → 10
i = 1 → 20 ...

Array Memory Representation

Array elements are stored in consecutive memory locations.



Example

If an int occupies 4 bytes, consecutive int elements are typically 4 bytes apart.

Concept

Contiguous storage makes array traversal efficient.

Reading Array Elements

Use a loop with scanf() to accept values from the user.

```
int a[5];

for (int i = 0; i < 5; i++)
{
    printf("Enter a[%d]: ", i);
    scanf("%d", &a[i]);
}
```

- Loop controls how many values are read.
- &a[i] gives the address of the current element.
- Each input is stored in a different array position.

Writing / Displaying Array Elements

Use printf() inside a loop to display each element.

```
int a[5] = {12, 25, 31, 44, 56};

for (int i = 0; i < 5; i++)
{
    printf("%d ", a[i]);
}
```



Output

12 25 31 44 56

Reading and Writing: Complete Program

A basic array input/output program.

```c
#include <stdio.h>
int main()
{
    int a[5], i;
    for(i=0; i<5; i++)
        scanf("%d", &a[i]);

    printf("Array: ");
    for(i=0; i<5; i++)
        printf("%d ", a[i]);
    return 0;
}
```

Step 1

Declare the array.

Step 2

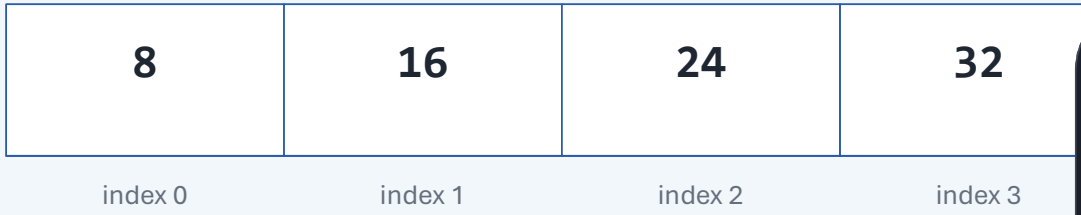
Read values using a loop.

Step 3

Display values using a loop.

Traversing an Array

Traversal means visiting every element, usually from first to last.




```
for (i = 0; i < n; i++)
{
    // process a[i]
}
```

- Start at index 0.
- Continue while i < n.
- Increment i after each iteration.
- The same pattern supports reading, writing, searching and calculations.

Example: Sum of Array Elements

```
int a[5] = {10,20,30,40,50};
int sum = 0;

for (int i = 0; i < 5; i++)
    sum = sum + a[i];

printf("Sum = %d", sum);
```

Process

sum starts at 0 and accumulates each element.

Calculation

10 + 20 + 30 + 40 + 50 = 150

Output

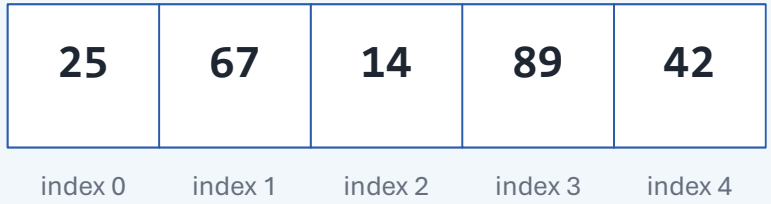
Sum = 150

Example: Find the Largest Element

```
int a[5] = {25, 67, 14, 89, 42};
int max = a[0];

for (int i = 1; i < 5; i++)
{
    if (a[i] > max)
        max = a[i];
}

printf("Max = %d", max);
```



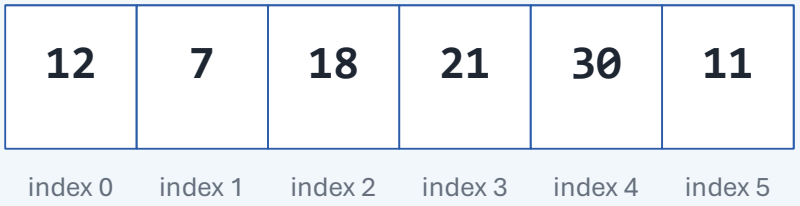
Result

Largest element = 89

Example: Count Even Numbers

```
int a[6] = {12, 7, 18, 21, 30, 11};
int count = 0;

for (int i = 0; i < 6; i++)
{
    if (a[i] % 2 == 0)
        count++;
}
```



Even values

12, 18, 30

Count

3

Reading n Elements Dynamically

A common pattern is to read the number of elements first.

```
int a[100], n, i;

printf("Enter n: ");
scanf("%d", &n);

for(i = 0; i < n; i++)
    scanf("%d", &a[i]);
```

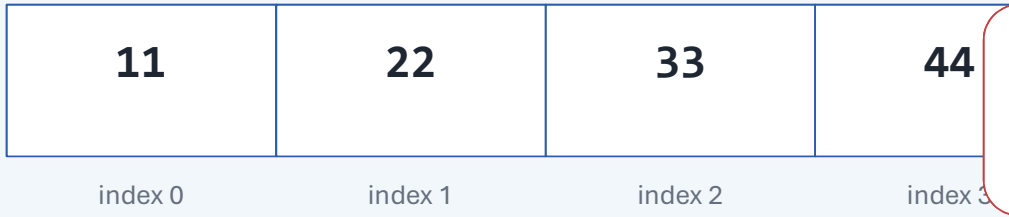
- The declared size is the maximum capacity.
- n tells the program how many elements are currently used.
- Always ensure n does not exceed the declared size.

Example

a[100] can safely hold at most 100 elements.

Array Bounds

Never access outside the valid index range.



Invalid

a[5] is outside an array declared as int a[5].

Why it matters

C does not automatically stop you from accessing an invalid index.

Valid

a[0], a[1], a[2], a[3], a[4]

Best practice

Use loops with correct bounds: i < size.

Common Mistakes

Watch for these errors while writing array programs.

1. Wrong index

Using a[5] for int a[5] instead of the last valid element a[4].

2. Wrong scanf()

scanf("%d", a[i]) is wrong; use scanf("%d", &a[i]).

3. Wrong loop condition

Using i <= size accesses one extra element.

4. Type mismatch

The array element type should match the input/output format.

5. Uninitialized values

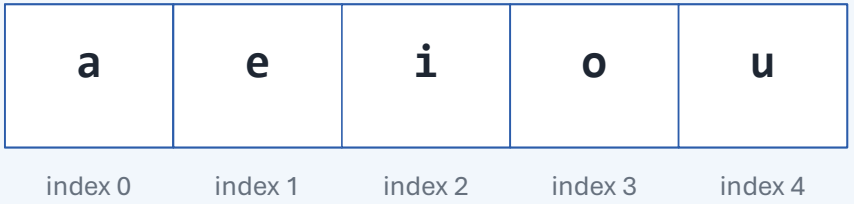
Reading or using array elements before giving them meaningful values can produce unexpected results.

Character Arrays: A Quick Look

Arrays can store characters too.

```
char vowels[5] = {'a','e','i','o','u'};

for (int i = 0; i < 5; i++)
    printf("%c ", vowels[i]);
```



Output

a e i o u

Note

Strings are also represented using character arrays.

Array Input: Marks Example

Read marks for 5 subjects and display them.

```
int marks[5], i;

for(i = 0; i < 5; i++)
{
    printf("Enter mark %d: ", i+1);
    scanf("%d", &marks[i]);
}

printf("Marks: ");
for(i = 0; i < 5; i++)
    printf("%d ", marks[i]);
```

- Notice the difference between index and displayed subject number.
- i starts at 0 because C arrays are zero-indexed.
- i+1 is used only for user-friendly numbering.

Practice Program: Average of n Numbers

```
int a[100], n, i;
float sum = 0, avg;

scanf("%d", &n);
for(i=0; i<n; i++)
{
    scanf("%d", &a[i]);
    sum += a[i];
}
avg = sum / n;
printf("Average = %.2f", avg);
```

Input

n followed by n numbers.

Processing

Store each number and accumulate sum.

Output

Average = sum / n

Quick Review

Key points to remember

- An array stores multiple values of the same type.
- A 1-D array uses one index.
- C array indexing starts at 0.
- Declaration specifies type, name and size.
- Initialization can be done at declaration.
- Loops are commonly used to read and display arrays.
- For size n, valid indices are 0 to n-1.
- Avoid out-of-bounds access.

Classroom Practice Questions

Try these programs before moving to the next topic.

1. Basic I/O

Read 10 integers into an array and display them in the same order.

2. Reverse Display

Read n integers and display them from last element to first.

3. Sum & Average

Find the sum and average of all elements.

4. Maximum & Minimum

Find the largest and smallest element in an array.

5. Even & Odd

Count how many elements are even and how many are odd.

6. Search

Read a key value and check whether it exists in the array.

Summary & Takeaway

Arrays make repeated data easy to store, access and process.



Declaration

int a[6];

Initialization

int a[6] = {15,28,41,54,67,80};

Read

scanf("%d", &a[i]);

Write

printf("%d", a[i]);

TWO-DIMENSIONAL ARRAYS IN C

Declaration • Initialization • Reading • Writing

	j=0	j=1	j=2
i=0	10	20	30
i=1	40	50	60
i=2	70	80	90

Learning Objectives

After this lesson, students should be able to:

- Explain the purpose of a two-dimensional array.
- Understand rows, columns and indices.
- Declare and initialize 2-D arrays in C.
- Read matrix elements using nested loops.
- Display/write elements using nested loops.
- Develop basic matrix-processing programs.

What is a Two-Dimensional Array?

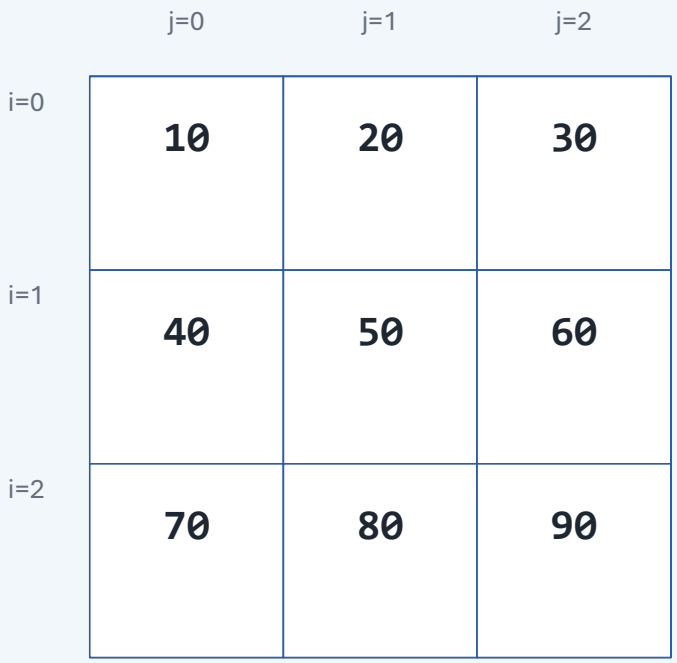
A 2-D array stores data in rows and columns.

Definition

A two-dimensional array is an array of arrays. It is commonly used to represent tables and matrices.

Structure

Each element is identified using two indices: row index and column index.



Why Do We Need 2-D Arrays?

Marks Table

Rows can represent students and columns can represent subjects.

Matrix

Mathematical matrices can be stored directly.

Grid Data

Useful for game boards, images, maps and tabular data.

	j=0	j=1	j=2
i=0	85	78	91
i=1	72	88	80
i=2	95	84	89

Rows and Columns

Understanding the two dimensions is essential.

	j=0	j=1	j=2	j=3
i=0	11	12	13	14
i=1	21	22	23	24
i=2	31	32	33	34

Rows

Horizontal groups of elements. Here there are 3 rows.

Columns

Vertical groups of elements. Here there are 4 columns.

Total elements

Rows × Columns = 3 × 4 = 12 elements.

2-D Array Syntax

General form of declaration

```
data_type array_name[rows][columns];

Examples:
int matrix[3][4];
float marks[5][3];
char board[8][8];
```

First dimension

Specifies the number of rows.

Second dimension

Specifies the number of columns.

Capacity

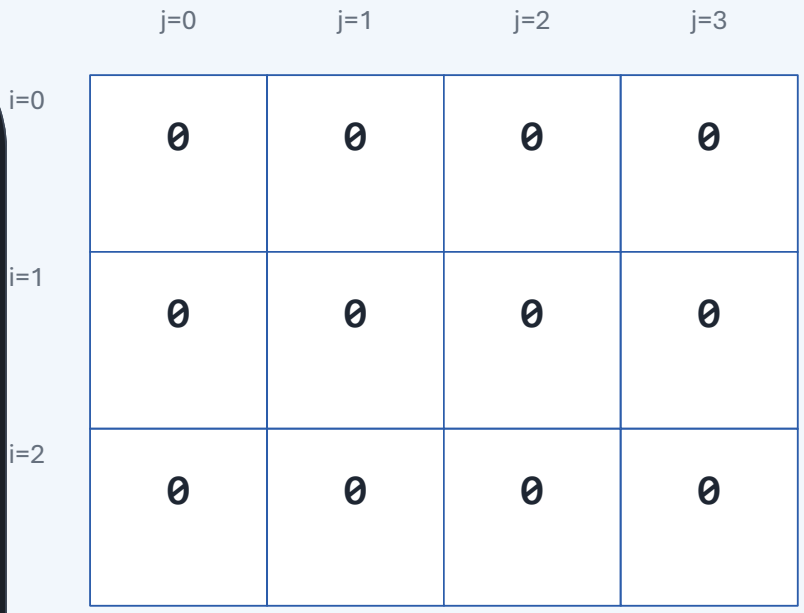
matrix[3][4] contains 3 × 4 = 12 elements.

Declaration Example

Let us declare a 3 × 4 integer matrix.

```
int a[3][4];

// 3 rows
// 4 columns
// Total = 12 integers
```



Valid indices

Rows: 0–2 | Columns: 0–3

Indexing a 2-D Array

Every element is accessed as array[row][column].

	j=0	j=1	j=2
i=0	10	20	30
i=1	40	50	60
i=2	70	80	90

```
a[0][0] = 10;
a[1][2] = 60;
a[2][1] = 80;
```

Remember

Both row and column indices start from 0.

Valid Index Range

For int a[3][4], what indices are valid?

	j=0	j=1	j=2	j=3
i=0	1	2	3	4
i=1	5	6	7	8
i=2	9	10	11	12

Rows

0, 1, 2

Columns

0, 1, 2, 3

Last element

a[2][3]

Initializing a 2-D Array

Values can be supplied during declaration.

```
int a[2][3] = {
    {10, 20, 30},
    {40, 50, 60}
};
```

	j=0	j=1	j=2
i=0	10	20	30
i=1	40	50	60

Structure

Each inner list represents one row.

Initialization Without Row Braces

C also permits a flat initializer list.

```
int a[2][3] = {
    10, 20, 30,
    40, 50, 60
};
```

	j=0	j=1	j=2
i=0	10	20	30
i=1	40	50	60

Tip

Row braces make the structure easier to read.

Partial Initialization

What happens when fewer values are supplied?

```
int a[2][3] = {
    {10, 20},
    {30}
};
```

	j=0	j=1	j=2
i=0	10	20	0
i=1	30	0	0

Rule

Unspecified elements are initialized to zero.

Omitting the First Dimension

The number of rows can be inferred from the initializer.

```
int a[][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```



Important

The column size must still be known.

Nested Loops and 2-D Arrays

A nested loop is the natural way to process rows and columns.

```
for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        // process a[i][j]
    }
}
```

i=0

i=1

i=2

j=0	j=1	j=2
1	2	3
4	5	6
7	8	9

Inner loop

Controls columns.

Outer loop

Controls rows.

Reading a 2-D Array

Use nested loops with scanf().

```
int a[3][4];

for (int i=0; i<3; i++)
{
    for (int j=0; j<4; j++)
        scanf("%d", &a[i][j]);
}
```

- i identifies the current row.
- j identifies the current column.
- &a[i][j] gives the address where the input is stored.
- Every element is read exactly once.

Writing / Displaying a 2-D Array

Use nested loops with printf().

```
for (int i=0; i<3; i++)
{
    for (int j=0; j<4; j++)
        printf("%d ", a[i][j]);
    printf("\n");
}
```

i=0

i=1

i=2

j=0	j=1	j=2	j=3
10	20	30	40
50	60	70	80
90	100	110	120

Why printf("\n")?

It moves output to the next row.

Complete Program: Read and Display Matrix

A basic 3 × 3 matrix input/output program.

```c
#include <stdio.h>
int main()
{
 int a[3][3], i, j;
 for(i=0;i<3;i++)
   for(j=0;j<3;j++)
     scanf("%d", &a[i][j]);

 for(i=0;i<3;i++) {
   for(j=0;j<3;j++)
     printf("%d ", a[i][j]);
   printf("\n");
 }
 return 0;
}
```

Phase 1

Read all elements.

Phase 2

Traverse row by row.

Phase 3

Print each row on a new line.

Row-Wise Traversal

The standard nested loop processes one complete row at a time.

	j=0	j=1	j=2
i=0	**11**	**12**	**13**
i=1	**21**	**22**	**23**
i=2	**31**	**32**	**33**

```
for(i=0; i<3; i++)
{
    for(j=0; j<3; j++)
        printf("%d ", a[i][j]);
    printf("\n");
}
```

Order

11 → 12 → 13 → 21 → 22 → 23 ...

Column-Wise Traversal

Swap the roles of the loops when you want columns first.

	j=0	j=1	j=2
i=0	11	12	13
i=1	21	22	23
i=2	31	32	33

```
for(j=0; j<3; j++)
{
    for(i=0; i<3; i++)
        printf("%d ", a[i][j]);
    printf("\n");
}
```

Order

11 → 21 → 31 → 12 → 22 → 32 ...

Example: Sum of All Matrix Elements

```
int sum = 0;

for(i=0; i<3; i++)
    for(j=0; j<3; j++)
        sum += a[i][j];

printf("Sum = %d", sum);
```

	j=0	j=1	j=2
i=0	1	2	3
i=1	4	5	6
i=2	7	8	9

Result

1+2+...+9 = 45

Example: Row Sum

Calculate the sum of each row separately.

```
for(i=0; i<3; i++)
{
    int sum = 0;
    for(j=0; j<4; j++)
        sum += a[i][j];
    printf("Row %d = %d\n", i, sum);
}
```

	j=0	j=1	j=2	j=3
i=0	10	20	30	40
i=1	5	15	25	35
i=2	2	4	6	8

Row sums

100, 80, 20

Example: Column Sum

Calculate the sum of each column.

```
for(j=0; j<4; j++)
{
    int sum = 0;
    for(i=0; i<3; i++)
        sum += a[i][j];
    printf("Col %d = %d\n", j, sum);
}
```

	j=0	j=1	j=2	j=3
i=0	10	20	30	40
i=1	5	15	25	35
i=2	2	4	6	8

Column sums

17, 39, 61, 83

Example: Matrix Addition

Two matrices of the same dimensions can be added element by element.

```
c[i][j] = a[i][j] + b[i][j];

for(i=0; i<2; i++)
  for(j=0; j<2; j++)
    c[i][j] = a[i][j] + b[i][j];
```

A

1 2
3 4

B

5 6
7 8

C = A + B

6 8
10 12

Example: Main Diagonal

For a square matrix, the main diagonal has equal row and column indices.

	j=0	j=1	j=2
i=0	5	2	1
i=1	3	8	4
i=2	7	6	9

```
for(i=0; i<3; i++)
    printf("%d ", a[i][i]);
```

Diagonal elements

a[0][0], a[1][1], a[2][2] → 5, 8, 9

Memory Representation

C stores 2-D arrays in row-major order.

	j=0	j=1	j=2
i=0	10	20	30
i=1	40	50	60

```
Memory order:
10 → 20 → 30 →
40 → 50 → 60
```

Row-major order

Elements of the first row are followed by elements of the second row.

Common Mistakes in 2-D Arrays

1. Missing second index

a[i] is incomplete when a is a 2-D array. Use a[i][j].

2. Wrong scanf()

Use &a[i][j] when reading an element.

3. Wrong bounds

For a[3][4], use i < 3 and j < 4.

4. Missing newline

Without \n after each row, output may appear as one long line.

5. Dimension mismatch

Matrix operations such as addition generally require compatible dimensions.

Practice Program: Matrix Input and Output

```
Task:
Read a 3 × 3 matrix from the user
and display it in matrix form.

Requirements:
1. Use int a[3][3]
2. Use nested loops for input
3. Use nested loops for output
4. Print each row on a new line
```



Expected format

```
1 2 3
4 5 6
7 8 9
```

Practice Programs

Try these programs using 2-D arrays.

1. Matrix Sum

Find the sum of all elements.

2. Row Sums

Find the sum of every row.

3. Column Sums

Find the sum of every column.

4. Matrix Addition

Add two compatible matrices.

5. Diagonal Sum

Find the main diagonal sum of a square matrix.

6. Transpose

Display rows as columns.

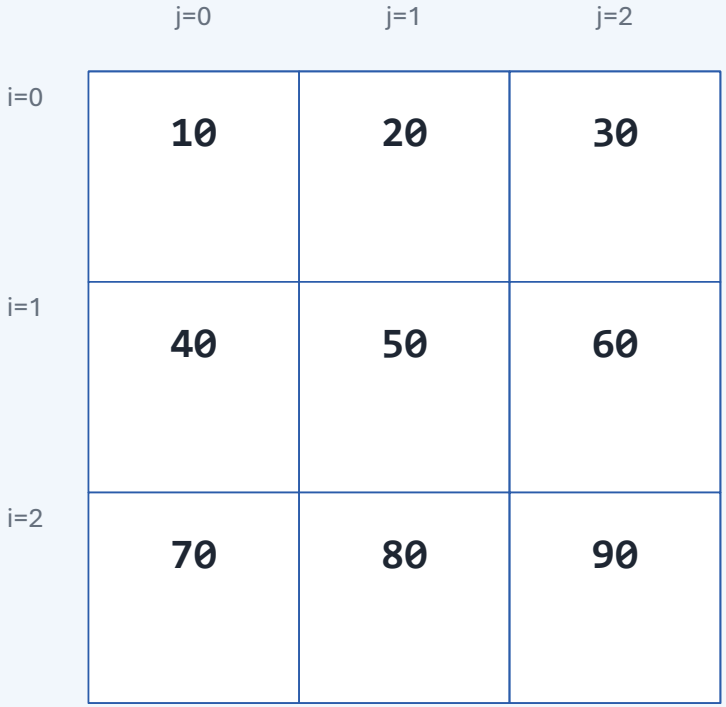
Quick Review

Important concepts from today's lesson

- A 2-D array represents data in rows and columns.
- Declaration: data_type a[rows][columns];
- Two indices are required: a[i][j].
- Both indices start from 0.
- Nested loops are used for reading and writing.
- Row-wise traversal uses the row loop outside.
- Column-wise traversal reverses the loop priority.
- For a[3][4], there are 12 elements.

Summary & Takeaway

Think of a 2-D array as a table stored systematically in memory.



Declaration

int a[3][3];

Initialization

int a[3][3] = {{10,20,30},{40,50,60},{70,80,90}};

Read

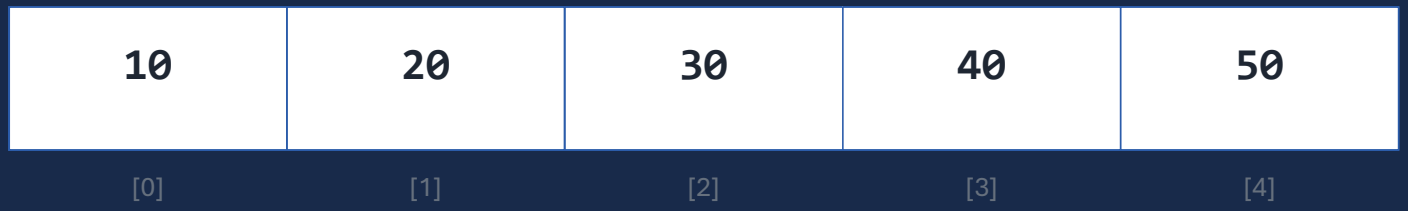
scanf("%d", &a[i][j]);

Write

printf("%d", a[i][j]);

MANIPULATING ELEMENTS OF ARRAYS

Access • Update • Insert • Delete • Search • Sort • Reverse



Learning Objectives

After this lesson, students should be able to:

- Access individual array elements using indices.
- Update and replace existing values.
- Traverse arrays using loops.
- Insert and delete elements by shifting values.
- Search, count and analyze elements.
- Reverse, copy, merge and compare arrays.
- Sort arrays in ascending and descending order.

What Does Array Manipulation Mean?

Manipulation means changing, examining or rearranging array elements.

Access

Read an element using its index.

Update

Replace an existing element with a new value.

Rearrange

Move, reverse or sort elements.

Insert/Delete

Add or remove an element by shifting values.

Search

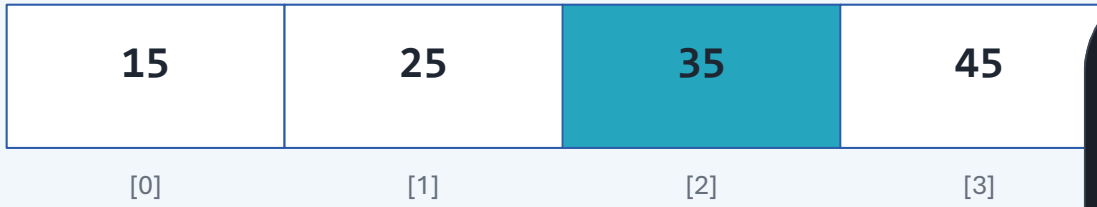
Find whether a particular value exists.

Process

Calculate sum, maximum, minimum, frequency, etc.

Accessing an Array Element

Use the index to access a specific element.




```
int a[5] = {15,25,35,45,55};

printf("%d", a[2]);

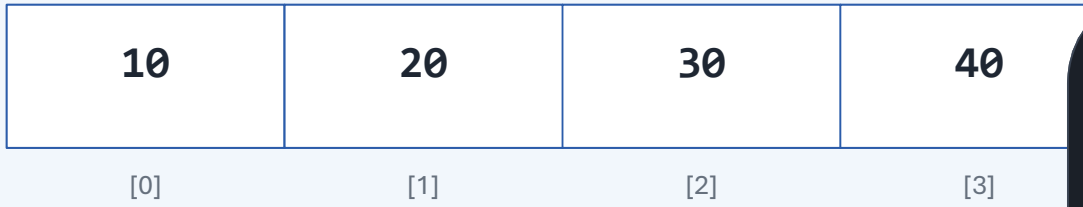
// Output: 35
```

Key point

Indexing starts at 0 in C.

Updating an Array Element

An existing element can be changed directly.




```
a[2] = 99;

Before: 10 20 30 40 50
After : 10 20 99 40 50
```

Operation

Only a[2] is replaced; other elements remain unchanged.

Updating Multiple Elements

Loops can modify many elements efficiently.

```
for (int i = 0; i < 5; i++)
{
    a[i] = a[i] * 2;
}
```



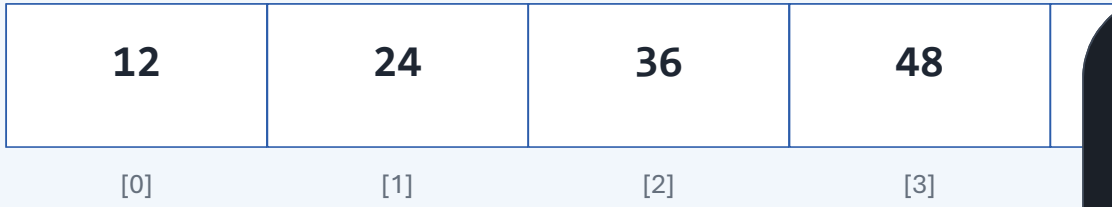


Example

Every element is multiplied by 2.

Traversing an Array

Traversal means visiting every element one by one.

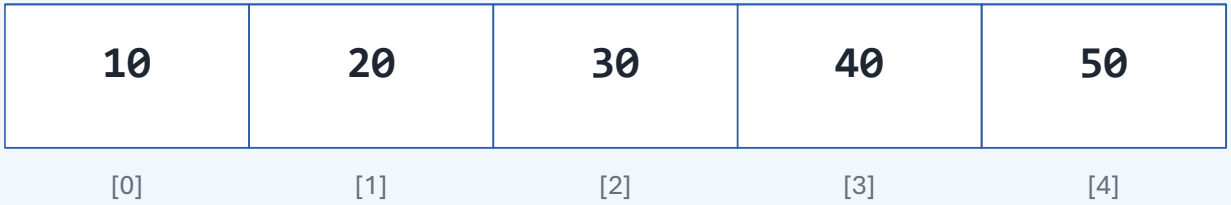



```
for (int i = 0; i < 5; i++)
{
    printf("%d ", a[i]);
}
```

- Starts at index 0.
- Stops before index size.
- The same pattern supports many array operations.

Inserting an Element

Insertion requires shifting elements to create an empty position.

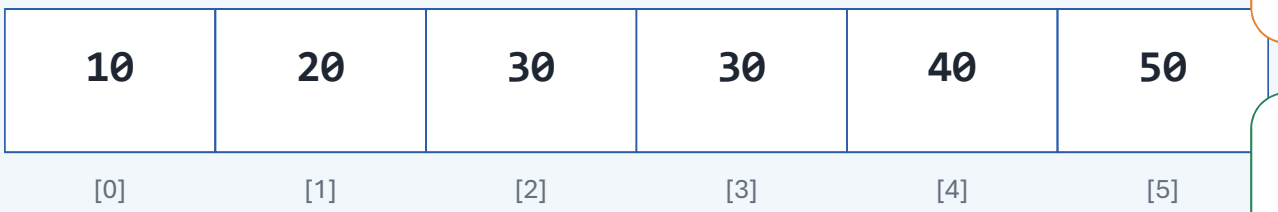


Goal

Insert 25 at index 2.

Shift

Move 50, 40 and 30 one position right.



Result

10 20 25 30 40 50

Insertion Algorithm

Work from the end toward the insertion position.

```
pos = 2;
value = 25;

for (i = n; i > pos; i--)
    a[i] = a[i-1];

a[pos] = value;
n++;
```

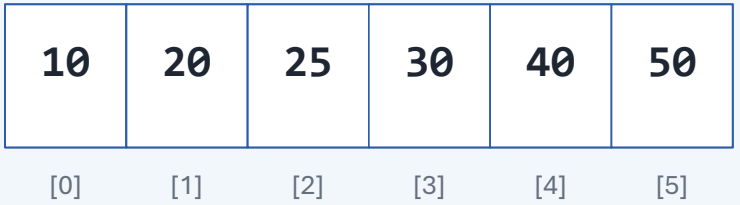
- Start shifting from the last used element.
- Move each element one position right.
- Place the new value at pos.
- Increase the logical size n.

Insertion Example in C

Insert an element at a specified position.

```
int a[10] = {10,20,30,40,50};
int n = 5, pos = 2, value = 25;

for(i=n; i>pos; i--)
    a[i] = a[i-1];
a[pos] = value;
n++;
```

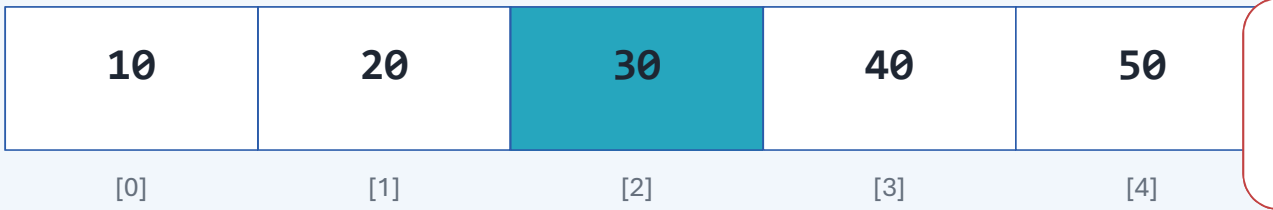


Output

10 20 25 30 40 50

Deleting an Element

Deletion requires shifting elements left to fill the gap.

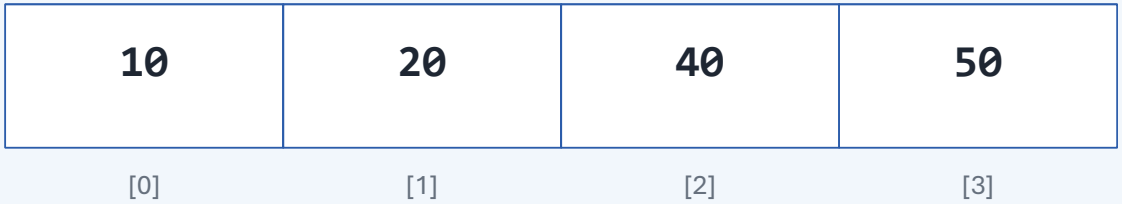


Goal

Delete the element at index 2.

Shift

Move later elements one position left.



Result

10 20 40 50

Deletion Algorithm

Start shifting from the deleted position.

```
pos = 2;

for (i = pos; i < n-1; i++)
    a[i] = a[i+1];

n--;
```

- The value at pos is overwritten by the next element.
- Continue until the last used element is reached.
- Decrease the logical size n.
- The physical array capacity does not change.

Searching an Array

Searching means checking whether a target value exists.




```
key = 48;

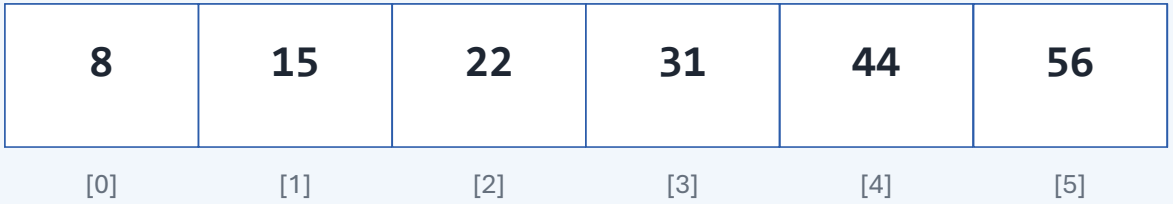
for(i=0; i<n; i++)
{
    if(a[i] == key)
    {
        printf("Found");
        break;
    }
}
```

Result

48 is found at index 3.

Linear Search

Linear search checks elements from left to right.



Step 1

Compare key with a[0].

Step 2

Continue until match is found.

Step 3

If no match, report not found.

```
for(i=0; i<n; i++)
    if(a[i] == key)
        return i;
return -1;
```

Finding Maximum and Minimum

Compare each element with the current best value.

```
max = a[0];
min = a[0];

for(i=1; i<n; i++)
{
    if(a[i] > max) max = a[i];
    if(a[i] < min) min = a[i];
}
```



Result

Maximum = 73 | Minimum = 18

Sum and Average

Manipulation can also mean calculating aggregate information.

```
sum = 0;
for(i=0; i<n; i++)
    sum += a[i];

average = (float)sum / n;
```



Calculation

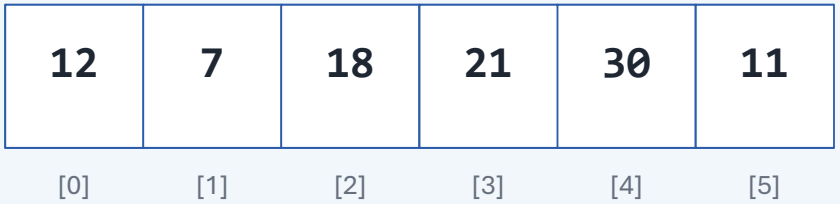
Sum = 150
Average = 30.0

Counting Even and Odd Elements

Use the modulus operator to classify values.

```
even = odd = 0;

for(i=0; i<n; i++)
{
    if(a[i] % 2 == 0)
        even++;
    else
        odd++;
}
```

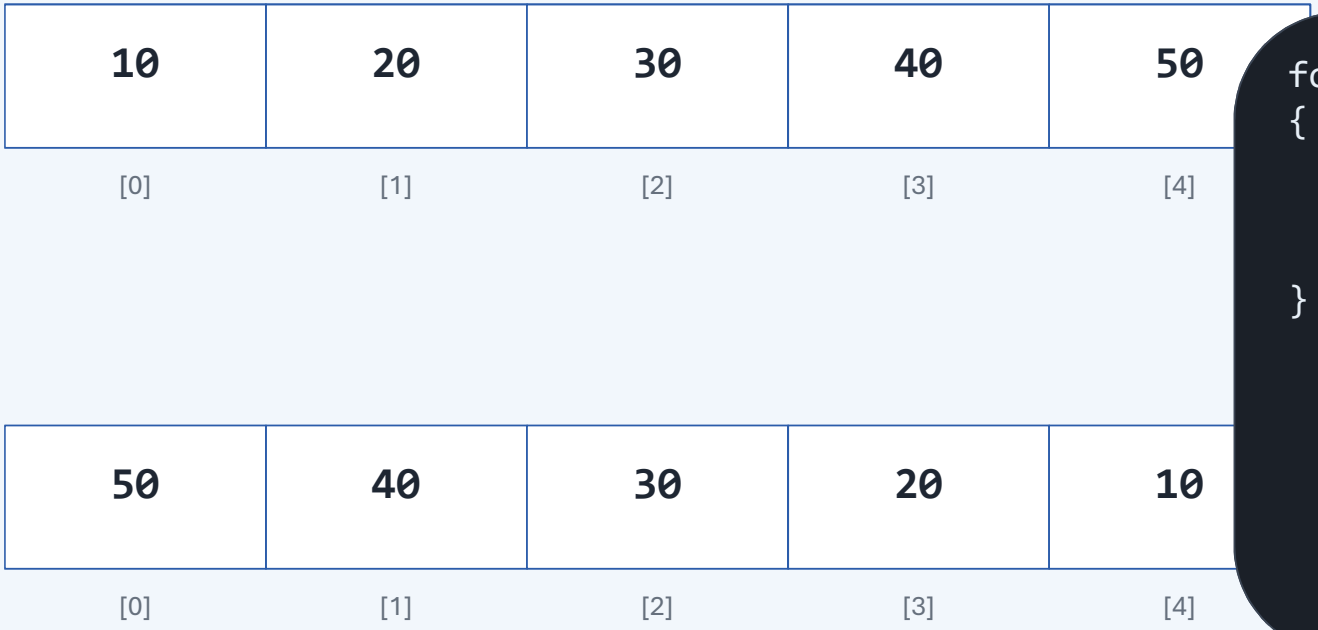


Result

Even = 3 | Odd = 3

Reversing an Array

Swap the first element with the last, then move inward.




```
for(i=0; i<n/2; i++)
{
    temp = a[i];
    a[i] = a[n-1-i];
    a[n-1-i] = temp;
}
```

Swapping Two Elements

A temporary variable prevents loss of a value.

```
temp = a[i];
a[i] = a[j];
a[j] = temp;
```



Use in

Reversing, sorting and rearranging arrays.

Copying an Array

Copy each element from source to destination.

```
int a[5] = {10,20,30,40,50};
int b[5];

for(i=0; i<5; i++)
    b[i] = a[i];
```



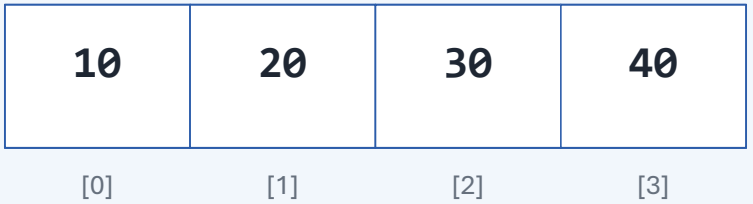
Comparing Two Arrays

Compare corresponding elements.

```
equal = 1;

for(i=0; i<n; i++)
{
    if(a[i] != b[i])
    {
        equal = 0;
        break;
    }
}
```



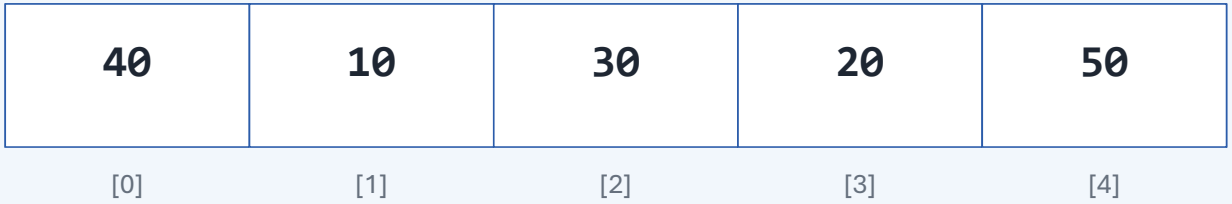


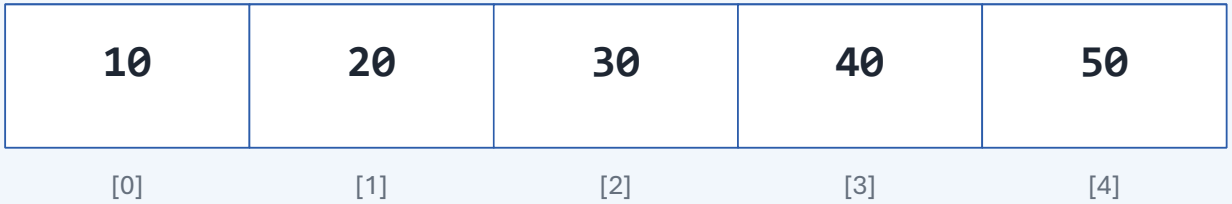
Result

Arrays are equal.

Sorting an Array

Sorting rearranges elements into a chosen order.





Ascending

Smallest → Largest

Descending

Largest → Smallest

Why sort?

Searching and reporting data can become easier.

Bubble Sort: Basic Idea

Repeatedly compare adjacent elements and swap them when they are out of order.

```
for(i=0; i<n-1; i++)
{
    for(j=0; j<n-1-i; j++)
    {
        if(a[j] > a[j+1])
        {
            temp=a[j];
            a[j]=a[j+1];
            a[j+1]=temp;
        }
    }
}
```



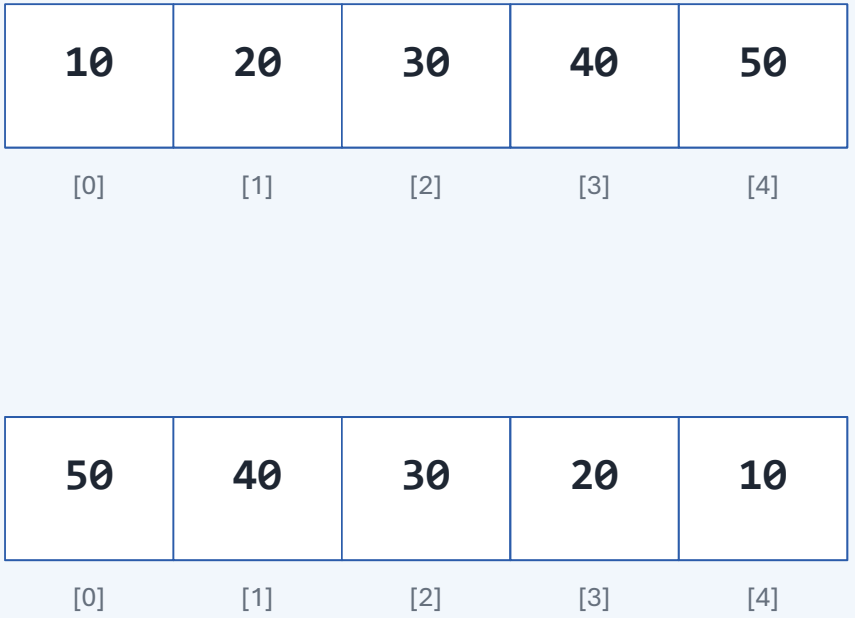
Key idea

Largest values gradually move toward the end.

Descending Order

Change the comparison condition to reverse the ordering.

```
if(a[j] < a[j+1])
{
    temp = a[j];
    a[j] = a[j+1];
    a[j+1] = temp;
}
```



Rule

For descending order, larger values should come first.

Frequency of an Element

Count how many times a value occurs.




```
key = 5;
count = 0;

for(i=0; i<n; i++)
    if(a[i] == key)
        count++;
```

Result

5 occurs 3 times.

Merging Two Arrays

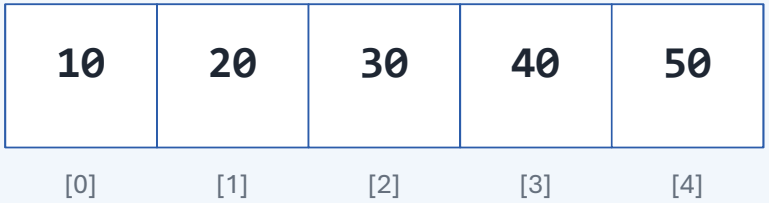
Combine elements of two arrays into a third array.

```
for(i=0; i<n1; i++)
    c[i] = a[i];

for(j=0; j<n2; j++)
    c[n1+j] = b[j];
```







Result

A + B → C

Common Mistakes

Avoid these errors while manipulating arrays.

Out-of-bounds access

Never access an index outside 0 to n-1.

Wrong shift direction

Insertion shifts right; deletion shifts left.

Losing a value

Use a temporary variable when swapping.

Incorrect logical size

After insertion n++; after deletion n--.

Unsorted assumptions

Do not use binary-search logic on an unsorted array unless it has first been sorted.

Integrated Example

Menu-style array manipulation program

```
1. Display
2. Insert
3. Delete
4. Search
5. Sort
6. Reverse
7. Exit

Use switch(choice) and
separate functions for each operation.
```

- Read n and array elements.
- Display the current array after each operation.
- Keep n updated after insertion/deletion.
- Validate positions before modifying the array.

Practice Problems

Apply the manipulation techniques.

1. Update

Replace every negative element with 0.

2. Insert

Insert a value at a user-specified position.

3. Delete

Delete the element at a given index.

4. Search

Find the first occurrence of a key.

5. Reverse

Reverse an array without using a second array.

6. Sort

Sort an array in ascending and descending order.

Summary & Key Takeaways

Array manipulation is built from indexing, loops and careful shifting.

Access / Update

a[i] reads or changes an element.

Insert / Delete

Shift elements right or left.

Search

Compare elements with a key.

Reverse

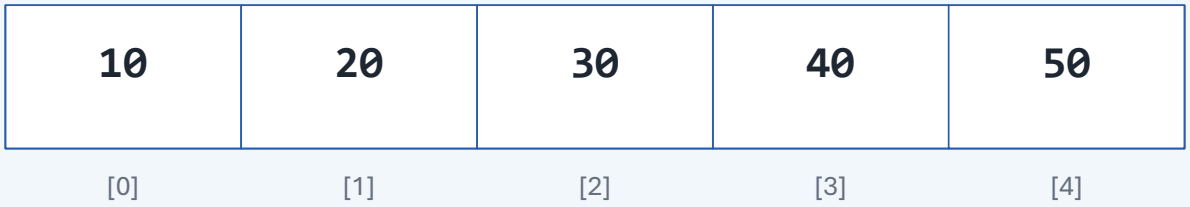
Swap symmetric elements.

Sort

Rearrange values into an order.

Process

Calculate sums, counts, min/max, etc.



STRINGS IN C

Introduction • Declaration • Initialization • Reading • Writing



Learning Objectives

After this lesson, students should be able to:

- Explain what a string means in C.
- Understand strings as character arrays.
- Explain the purpose of the null character '\0'.
- Declare and initialize strings.
- Read strings using scanf() and fgets().
- Write strings using printf() and puts().
- Handle strings containing spaces correctly.

What is a String?

A string is a sequence of characters terminated by the null character '\0'.

Definition

In C, a string is stored in a character array and ends with the special character '\0'.



Important

The visible text has 5 characters, but the array needs space for 6 characters including '\0'.

Character vs String

A character and a string are not the same.

Character

A single character is written in single quotes.

Example: 'A'

String

A sequence of characters is written in double quotes.

Example: "APPLE"

```
char ch = 'A';
char name[] = "APPLE";
```

The Null Character '\0'

The null character marks the end of a C string.



String terminator

'\0' tells string-processing functions where the string ends.

Not the same as '0'

'0' is the character zero. '\0' is the null character with value zero.

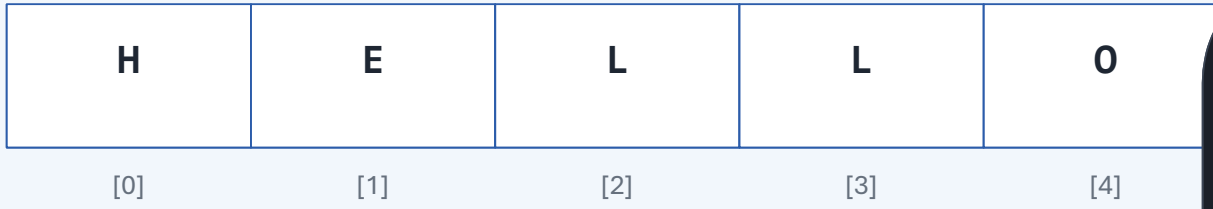



```
char word[2] = {'O', '\0'};

// Valid string: "O"
```

How a String is Stored

Each character occupies one array element.




```
char name[] = "HELLO";

// Memory concept
name[0] = 'H'
name[1] = 'E'
name[2] = 'L'
name[3] = 'L'
name[4] = 'O'
name[5] = '\0'
```

Array size

The compiler needs 6 character positions for "HELLO" when the size is inferred.

String Declaration

A string is declared as a character array.

```
char name[20];
char city[30];
char message[100];
```

Syntax

char string_name[size];

Capacity

The size is the maximum number of characters stored, including '\0' when used as a C string.

Example

char name[20] can store up to 19 visible characters plus '\0'.

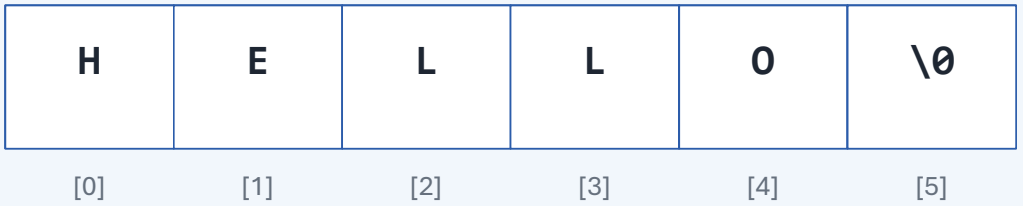
String Initialization

Strings can be initialized using a string literal.

```
char name[6] = "Girish";

char city[] = "Hyderabad";

char word[10] = "Hello";
```



Automatic sizing

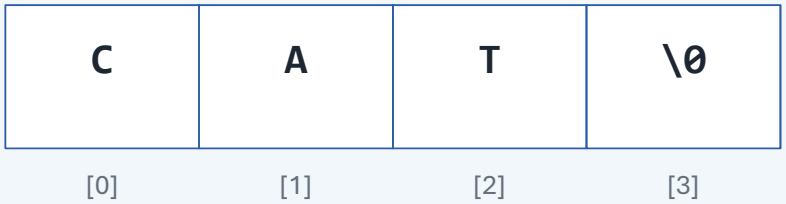
char city[] = "Hyderabad"; lets the compiler determine the required array size.

String Literal vs Character List

Both approaches can create a string.

```
char a[] = "CAT";

char b[] = {'C', 'A', 'T', '\0'};
```



Equivalent

Both arrays contain C, A, T and the terminating '\0'.

String Size and the Null Character

Always account for '\0' when manually specifying the size.

Correct

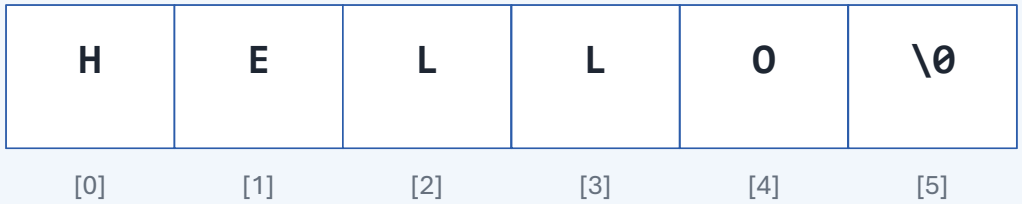
char name[6] = "HELLO";

Why 6?

5 visible characters + 1 null character.

Too small

char name[5] cannot store "HELLO" as a conventional C string.



Reading a String with scanf()

The %s conversion specifier reads a word.

```
char name[30];

printf("Enter name: ");
scanf("%29s", name);

printf("Name = %s", name);
```

No & before name

For %s, the array name already represents the address of its first element.

Width limit

%29s helps prevent reading more than the array can hold.

Limitation

scanf("%s") stops reading at whitespace.

Why scanf("%s") Does Not Read Spaces

This is one of the most common beginner mistakes.

```
char name[50];
scanf("%49s", name);

Input:
Girish Kumar

Stored:
"Girish"
```

What happens?

scanf("%s") reads characters until whitespace such as a space, tab or newline.

Result

Only the first word is stored.

Better option

Use fgets() when a string may contain spaces.

Reading a String with fgets()

fgets() is useful for reading a complete line.

```
char name[50];

printf("Enter name: ");
fgets(name, sizeof(name), stdin);

printf("Name = %s", name);
```

Advantage

Can read spaces within the input line.

Size control

sizeof(name) limits the number of characters read.

Newline note

fgets() may store the newline '\n' if there is room.

scanf() vs fgets()

Choose the input function based on the required input.

scanf("%s")

- Reads a word
- Stops at whitespace
- Useful for single-word input
- Example: username

fgets()

- Reads a line
- Can include spaces
- Controls maximum input size
- Example: full name

Rule of thumb

If the input can contain spaces, prefer fgets(). If it is a single word, scanf("%s") may be appropriate.

Writing a String with printf()

Use %s to print a string.

```
char name[] = "Girish";

printf("%s", name);

printf("Student: %s", name);
```

Format specifier

%s tells printf() that the argument is a string.

Output

Girish
Student: Girish



Writing a String with puts()

puts() prints a string followed by a newline.

```
char message[] = "Welcome to C";

puts(message);
```

Output

Welcome to C

Difference

puts() automatically moves the cursor to the next line.

Use case

Simple string output without formatting.

Complete Program: Read and Write a Word

A simple string input/output program.

```c
#include <stdio.h>
int main()
{
    char name[30];
    printf("Enter name: ");
    scanf("%29s", name);
    printf("Hello, %s!", name);
    return 0;
}
```

Input

One word such as Girish.

Storage

Stored in a character array.

Output

Hello, Girish!

Complete Program: Read a Full Name

Use fgets() when spaces are required.

```c
#include <stdio.h>
int main()
{
    char name[50];
    printf("Enter full name: ");
    fgets(name, sizeof(name), stdin);
    printf("Name: %s", name);
    return 0;
}
```

Input

Girish Kumar

Stored

Both words can be read.

Output

Name: Girish Kumar

Removing the Newline from fgets()

fgets() may store '\n', so it is often removed.

```
char name[50];
fgets(name, sizeof(name), stdin);

for(i=0; name[i]!='\0'; i++)
{
    if(name[i] == '\n')
        name[i] = '\0';
}
```

Before

"Girish Kumar\n\0"

After

"Girish Kumar\0"

Why?

Makes later string processing cleaner.

Accessing Individual Characters

A string is still a character array, so indexing works.




```
char word[] = "HELLO";

printf("%c", word[1]);
// E

word[4] = 'Y';
// HELLY
```

Remember

%c prints one character; %s prints a string.

Reading Multiple Strings

Arrays can store several strings using a 2-D character array.

```
char names[3][20];

for(i=0; i<3; i++)
    scanf("%19s", names[i]);

for(i=0; i<3; i++)
    printf("%s\n", names[i]);
```

Concept

Each row can hold one string.

Example

names[0], names[1], names[2]

Use

Store a list of student names.

Common String Mistake: Missing Space for '\0'

Always leave room for the terminator.

```
char word[5] = "HELLO";

// Problem:
// 5 letters + '\0' = 6 positions
```

Correct

char word[6] = "HELLO";

Better for flexibility

char word[] = "HELLO";

Rule

Size must be at least characters + 1.

Common String Mistake: Using = After Declaration

Character arrays cannot be assigned a string literal with = after declaration.

```
char name[20];

// Wrong:
name = "Girish";

// Use input functions
// or copy characters instead.
```

Declaration + initialization

char name[20] = "Girish";

After declaration

Use fgets(), scanf(), or a character-copying method.

Important

Arrays are not assignable like ordinary scalar variables.

String Input and Previous Input

Mixing scanf() and fgets() requires care.

```
int age;
char name[50];

scanf("%d", &age);
getchar();
fgets(name, sizeof(name), stdin);
```

The issue

scanf("%d") can leave the newline in the input buffer.

Common solution

Consume the leftover newline before calling fgets().

Example

getchar();

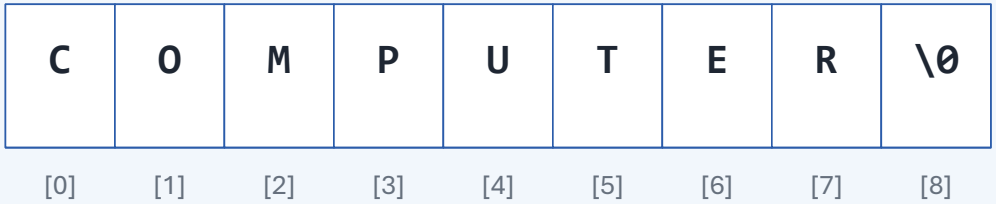
Example: Count Characters

Traverse the string until '\0'.

```
char word[] = "COMPUTER";
int count = 0;

while(word[count] != '\0')
    count++;

printf("Length = %d", count);
```



Result

Length = 8

Idea

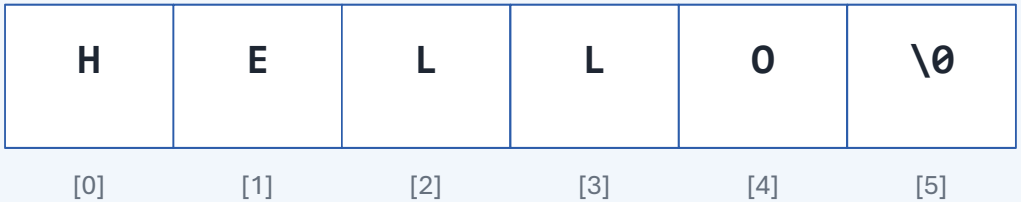
The null character marks where traversal stops.

Example: Display Characters One by One

A string can be processed character by character.

```
char word[] = "HELLO";

for(i=0; word[i]!='\0'; i++)
    printf("%c\n", word[i]);
```



Output

```
H
E
L
L
O
```

Example: Read and Display a Sentence

A practical example using fgets().

```
char sentence[100];

printf("Enter sentence: ");
fgets(sentence, sizeof(sentence), stdin);

printf("You entered:\n%s", sentence);
```

Input

C programming is interesting.

Function

fgets() reads the entire line.

Output

The sentence is displayed as entered.

Practice Programs

Try these string programs.

1. Read & Display

Read a student's name and display it.

2. Full Name

Read a full name containing spaces.

3. Character Count

Count characters without using strlen().

4. Vowels

Count vowels in a string.

5. Uppercase

Convert lowercase letters to uppercase.

6. Reverse

Display a string in reverse order.

Quick Review

Important concepts to remember

- A string is a sequence of characters ending with '\0'.
- Strings are stored in character arrays.
- Use double quotes for string literals.
- Use single quotes for individual characters.
- scanf("%s") normally reads one word.
- fgets() can read a line containing spaces.
- Use %s with printf() for string output.
- Remember the extra space required for '\0'.

Summary & Key Takeaways

Strings are character arrays with a special terminator.

Declaration

```
char name[30];
```

Initialization

```
char name[] = "Girish";
```

Read Word

```
scanf("%29s", name);
```

Read Line

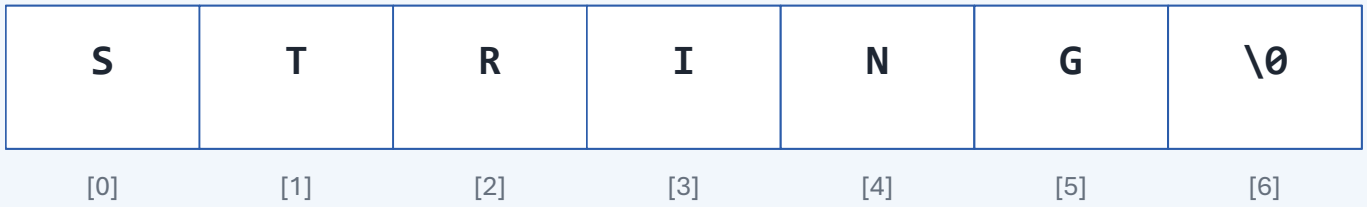
```
fgets(name, sizeof(name), stdin);
```

Write

```
printf("%s", name);
```

Write Line

```
puts(name);
```



STRING HANDLING FUNCTIONS IN C

<string.h> • Length • Copy • Concatenate • Compare • Search



Learning Objectives

After this lesson, students should be able to:

- Explain why string handling functions are needed.
- Use the <string.h> header file correctly.
- Find string length using strlen().
- Copy strings using strcpy() and strncpy().
- Join strings using strcat() and strncat().
- Compare strings using strcmp() and strncmp().
- Search characters and substrings using strchr(), strrchr() and strstr().
- Use tokenization and related functions appropriately.

Why String Handling Functions?

C does not provide string operations through the basic assignment operator.

Length

Find how many characters are in a string.

Copy

Transfer one string into another character array.

Concatenate

Join two strings together.

Compare

Determine the lexicographic relationship between strings.

Search

Find a character or substring.

Tokenize

Break a string into smaller tokens.

The <string.h> Header File

Most standard C string manipulation functions are declared in <string.h>.

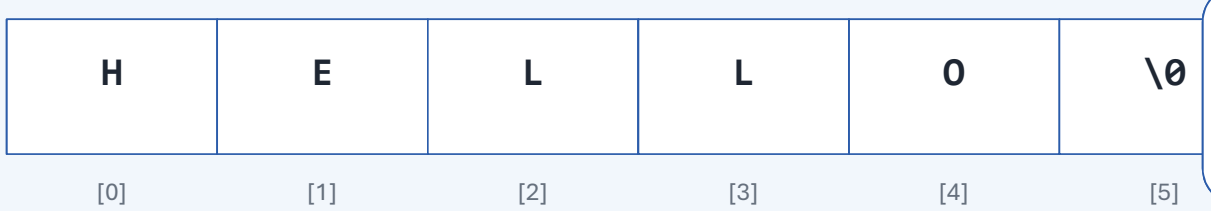
```c
#include <stdio.h>
#include <string.h>

int main()
{
    char name[] = "Girish";
    printf("%zu", strlen(name));
    return 0;
}
```

- Include <string.h> before using its functions.
- The header provides declarations for standard string/memory functions.
- Using the correct header improves type checking and portability.

Important String Concept

String functions depend on the terminating null character '\0'.



String ends at '\0'

Functions such as strlen(), strcpy() and strcmp() process characters until the terminator.

Not part of length

strlen("HELLO") returns 5, not 6.

Safety

A valid C string must have a null terminator.

Function Categories

String functions can be grouped according to their purpose.

Length

strlen()

Copy

strcpy(), strncpy()

Concatenate

strcat(), strncat()

Compare

strcmp(), strncmp()

Search

strchr(), strrchr(), strstr()

Tokenize

strtok()

Related memory functions

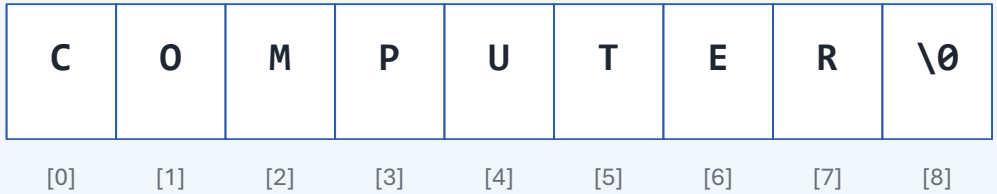
memcpy(), memmove(), memset(), memcmp() operate on bytes, not specifically strings.

strlen(): String Length

strlen() returns the number of characters before '\0'.

```
char word[] = "COMPUTER";

size_t n = strlen(word);
printf("Length = %zu", n);
```



Result

Length = 8

Important

The '\0' is not counted.

strlen(): Syntax and Return Type

Understand the function prototype.

```
size_t strlen(const char *str);
```

Parameter

A pointer to the string whose length is required.

Return value

Returns the number of characters before '\0'.

size_t

An unsigned integer type commonly used for sizes.

strlen() Example: Count Characters

```
char name[50];
fgets(name, sizeof(name), stdin);

// If newline is present,
// remove it first.

printf("Length = %zu", strlen(name));
```

Input

Girish Kumar

strlen()

Counts characters in the stored string.

Result

11 characters (without the newline).

strcpy(): Copying a String

strcpy() copies the source string, including '\0', into the destination.

```
char source[] = "Hello";
char dest[20];

strcpy(dest, source);

printf("%s", dest);
```

Syntax

strcpy(destination, source);

Direction

Data moves from source → destination.

Requirement

Destination must have enough space.

strcpy(): Visualizing the Copy

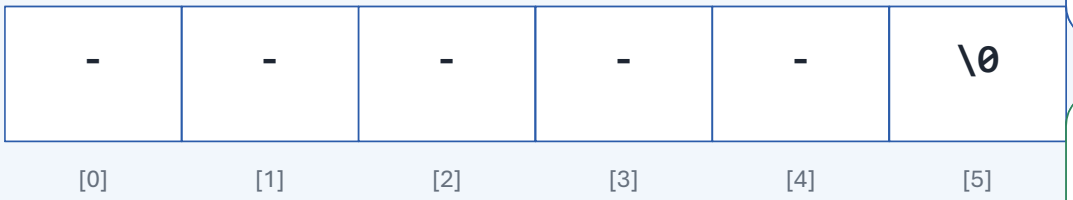


Before

dest contains uninitialized/old data.

Operation

strcpy(dest, source);



After

dest becomes "HELLO".

strcpy(): Complete Example

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char source[30] = "Programming";
    char copy[30];

    strcpy(copy, source);
    printf("%s", copy);
    return 0;
}
```

Source

Programming

Destination

copy

Output

Programming

strncpy(): Limited Copy

strncpy() copies at most n characters.

```
char source[] = "ABCDEFGHIJ";
char dest[6];

strncpy(dest, source, 5);
dest[5] = '\0';
```

Purpose

Useful when you want to limit how many characters are copied.

Important

strncpy() may not append '\0' when the source is long enough.

Best practice

Explicitly terminate the destination when necessary.

strcat(): Concatenating Strings

strcat() appends the source string to the destination.

```
char first[30] = "Hello ";
char second[] = "World";

strcat(first, second);

printf("%s", first);
```

Before

first = "Hello "

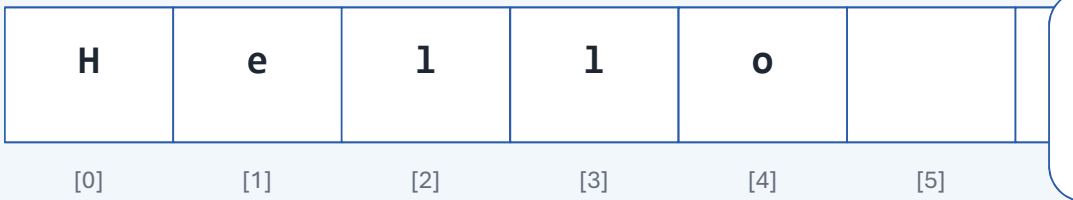
Operation

strcat(first, second);

After

first = "Hello World"

strcat(): Visualizing Concatenation





Source

"World" is appended.

Destination

The destination string is modified.

Result

"Hello World"

strncat(): Limited Concatenation

strncat() appends at most n characters.

```
char dest[30] = "Hello ";
char src[] = "Programming";

strncat(dest, src, 4);

printf("%s", dest);
```

Operation

Only the first 4 characters of src are appended.

Result

Hello Prog

Capacity

Destination still needs sufficient space.

strcmp(): Comparing Strings

strcmp() compares strings lexicographically.

```
int result;
result = strcmp("apple", "apple");

if(result == 0)
    printf("Equal");
```

Return = 0

The two strings are equal.

Return < 0

First string compares less than second.

Return > 0

First string compares greater than second.

strcmp(): Understanding the Result

Do not assume the result will always be exactly -1 or +1.

```
int r = strcmp("apple", "banana");

if (r == 0)
    printf("Equal");
else if (r < 0)
    printf("apple comes first");
else
    printf("banana comes first");
```

Correct logic

Check whether result is < 0, == 0, or > 0.

Why?

The exact non-zero value is implementation-dependent.

Case-sensitive

Uppercase and lowercase letters can compare differently.

strncmp(): Compare Limited Characters

strncmp() compares at most n characters.

```
char a[] = "Computer";
char b[] = "Compare";

if (strncmp(a, b, 3) == 0)
    printf("First 3 match");
```

Purpose

Useful when only a prefix of the strings should be compared.

Example

"Com" and "Com" match for the first 3 characters.

Caution

Choose n carefully for your intended comparison.

String Comparison: A Common Mistake

The == operator does not compare string contents.

```
char a[] = "Hello";
char b[] = "Hello";

// Wrong idea:
if (a == b)
    printf("Equal");

// Correct:
if (strcmp(a, b) == 0)
    printf("Equal");
```

Why == is wrong

It compares addresses/pointers rather than string contents.

Use strcmp()

Compare actual character sequences.

Exam tip

This is a very common C question.

strchr(): Find a Character

strchr() finds the first occurrence of a character.

```
char text[] = "PROGRAMMING";
char *p;

p = strchr(text, 'G');

if(p != NULL)
    printf("Found: %s", p);
```

Return value

Pointer to the first matching character, or NULL if not found.

Example

For 'G', p points to the G inside the string.

Useful for

Finding separators, vowels, punctuation, etc.

strrchr(): Find the Last Occurrence

strrchr() searches from the end and returns the last occurrence.

```
char text[] = "banana";
char *p = strrchr(text, 'a');

if(p != NULL)
    printf("%s", p);
```

First occurrence

strchr("banana", 'a') → first a

Last occurrence

strrchr("banana", 'a') → last a

Return

Pointer to the matching character.

strstr(): Find a Substring

strstr() searches for one string inside another.

```
char text[] = "C programming";
char *p;

p = strstr(text, "program");

if(p != NULL)
    printf("Found: %s", p);
```

Search

Finds the first occurrence of the substring.

Found

Returns a pointer to the matching position.

Not found

Returns NULL.

strstr(): Practical Example

Search for a word in a sentence.

```
char sentence[100];
fgets(sentence, sizeof(sentence), stdin);

if (strstr(sentence, "C") != NULL)
    printf("C is mentioned");
else
    printf("C not found");
```

Input

I am learning C programming.

Search

strstr(sentence, "C")

Result

C is mentioned

strtok(): Tokenizing a String

strtok() splits a string into tokens using delimiters.

```
char text[] = "C,Python,Java";
char *token;

token = strtok(text, ",");

while(token != NULL)
{
    printf("%s\n", token);
    token = strtok(NULL, ",");
}
```

Delimiter

,

Tokens

C | Python | Java

Important

strtok() modifies the input string.

strtok(): How It Works

The first call uses the string; later calls use NULL.

```
token = strtok(text, ",");
// first token

token = strtok(NULL, ",");
// next token

token = strtok(NULL, ",");
// next token
```

First call

Pass the original string.

Later calls

Pass NULL to continue the same tokenization.

Caution

Because strtok() modifies the string, do not use it on a string literal.

Related Memory Functions

These operate on raw memory/bytes and are useful with character arrays.

memcpy()

Copies n bytes from source to destination.

memmove()

Copies bytes safely when source and destination overlap.

memset()

Fills a memory region with a byte value.

memcmp()

Compares n bytes of two memory regions.

Important distinction

Memory functions do not automatically stop at '\0'. They work with the specified number of bytes.

```
memset(buffer, 0, sizeof(buffer));
```

String Function Summary Table

A quick reference for the most important functions.

Function	Purpose	Return type
`strlen(s)`	Find length	size_t
`strcpy(d,s)`	Copy string	char *
`strncpy(d,s,n)`	Limited copy	char *
`strcat(d,s)`	Append string	char *
`strncat(d,s,n)`	Limited append	char *
`strcmp(a,b)`	Compare strings	int
`strncmp(a,b,n)`	Limited compare	int
`strchr(s,c)`	Find character	char *
`strrchr(s,c)`	Find last character	char *
`strstr(s,sub)`	Find substring	char *
`strtok(s,delim)`	Tokenize	char *

Destination Size Matters

Many string bugs happen because the destination array is too small.

```
char src[] = "Programming";
char dest[5];

// Dangerous:
strcpy(dest, src);
```

Problem

The destination cannot hold the complete source string plus '\0'.

Consequence

Buffer overflow and undefined behavior may occur.

Rule

Always ensure the destination has sufficient capacity.

Safer Input and String Operations

Good string programming starts with controlling capacity.

Use width with scanf

scanf("%49s", name) for char name[50].

Prefer fgets for lines

fgets(name, sizeof(name), stdin) handles spaces and limits input.

Check destination capacity

Before strcpy/strcat, verify enough room exists.

Use bounded operations carefully

strncpy/strncat have rules that require careful handling of termination.

Modern note

For teaching standard C, focus on portable functions and correct buffer management rather than assuming every compiler provides non-standard alternatives.

Example: Copy and Compare User Input

A practical program using strlen(), strcpy() and strcmp().

```
char a[50], b[50], copy[50];

fgets(a, sizeof(a), stdin);
fgets(b, sizeof(b), stdin);

strcpy(copy, a);

if (strcmp(a, b) == 0)
    printf("Same");
else
    printf("Different");
```

strlen()

Can inspect the input length.

strcpy()

Makes a separate copy.

strcmp()

Checks content equality.

Example: Join First Name and Last Name

strcat() can construct a full name.

```
char first[50], last[30];

fgets(first, sizeof(first), stdin);
fgets(last, sizeof(last), stdin);

// remove newlines first
strcat(first, " ");
strcat(first, last);

printf("%s", first);
```

Input

Girish + Kumar

Operation

first ← first + " " + last

Output

Girish Kumar

Example: Search in a Sentence

Combine fgets() and strstr() for simple text searching.

```
char sentence[200];
char key[30];

fgets(sentence, sizeof(sentence), stdin);
fgets(key, sizeof(key), stdin);

if (strstr(sentence, key) != NULL)
    printf("Found");
else
    printf("Not found");
```

Sentence

The student studies C.

Key

studies

Result

Found

Example: Count Vowels Without Library Functions

String functions are useful, but loops can also process characters directly.

```
char s[100];
int count = 0;

fgets(s, sizeof(s), stdin);
for(i=0; s[i]!='\0'; i++)
{
    if(s[i]=='a'||s[i]=='e'||s[i]=='i'||
       s[i]=='o'||s[i]=='u')
        count++;
}
```

Purpose

Shows how string functions and manual traversal complement each other.

Practice

Try handling uppercase vowels too.

Output

Number of vowels.

String Handling Workflow

A useful mental model for solving string problems.

1. Input

Choose scanf() or fgets().

2. Validate

Check length and capacity.

3. Process

Use the appropriate string function.

4. Check result

Compare against 0 or NULL as appropriate.

5. Output

Use printf() or puts().

6. Test

Test empty, long and space-containing inputs.

Key principle

Know what the function returns and what it modifies.

Common Mistakes

These are frequent examination and programming errors.

1. Forgetting #include <string.h>

String function declarations are provided by this header.

2. Using == for strings

Use strcmp() to compare string contents.

3. Small destination

strcpy/strcat require enough destination space.

4. Misunderstanding strlen()

The '\0' is not counted.

5. scanf("%s") with spaces

It stops at whitespace; use fgets() for a full line.

6. Ignoring return values

strcmp/search functions communicate results through their return values.

Practice Programs – Basic

Try these programs using standard string functions.

1. Length

Read a string and find its length using strlen().

2. Copy

Copy one string into another using strcpy().

3. Join

Join first name and last name using strcat().

4. Compare

Read two strings and compare them using strcmp().

5. Search Character

Find whether a particular character occurs using strchr().

6. Search Word

Find a substring using strstr().

Practice Programs – Advanced

Combine multiple functions to solve practical problems.

1. Word Frequency

Count occurrences of a word in a sentence.

2. Tokenize

Split a comma-separated list using strtok().

3. Password Check

Compare entered password with a stored string.

4. Prefix Test

Use strncmp() to test whether two strings share a prefix.

5. Sentence Search

Read a sentence and search for a keyword.

6. Student Records

Store and process multiple names using 2-D character arrays.

Quick Review

Essential string handling functions

- strlen() → find string length.
- strcpy() / strncpy() → copy strings.
- strcat() / strncat() → concatenate strings.
- strcmp() / strncmp() → compare strings.
- strchr() / strrchr() → search for characters.
- strstr() → search for a substring.
- strtok() → split a string into tokens.
- Always consider '\0' and destination capacity.

Summary & Key Takeaways

String handling is about safely processing character arrays.

Length

strlen()

Copy

strcpy() / strncpy()

Join

strcat() / strncat()

Compare

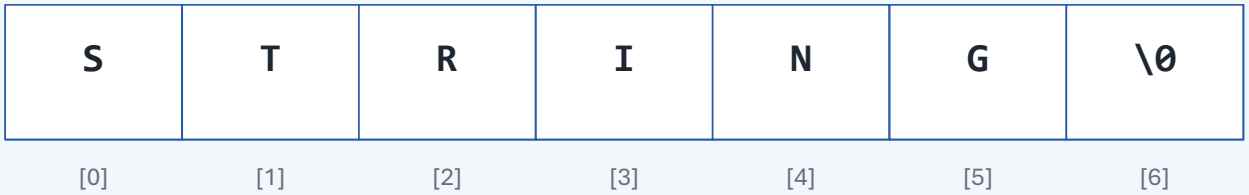
strcmp() / strncmp()

Search

strchr() / strstr()

Tokenize

strtok()



C PROGRAMMING

Handling Two-Dimensional Strings in C

Arrays of strings • Declaration • Initialization • Reading • Writing • Processing

Learning Objectives

- Explain how a 2-D character array stores multiple strings.
- Declare and initialize an array of strings.
- Read and display strings using loops.
- Access complete strings and individual characters.
- Use strlen(), strcpy(), strcmp(), and strcat() with individual rows.
- Search, sort, count, and process multiple strings.

One String vs Many Strings

One string

char name[20];
Stores ONE string.

Many strings

char names[5][20];
Stores 5 strings; each row has 20 char positions.

A 2-D character array can be viewed as a table of strings.

Meaning of a 2-D String

- C has no separate string-array type.
- A string is a char array ending with '\0'.
- A 2-D char array provides several rows of character storage.
- Each row can be treated as one C string.
- Example: char names[4][20];

Declaration Syntax

ROWS

Number of strings that can be stored.

COLS

Maximum character capacity of each row.

A 2-D character array can be viewed as a table of strings.

Example

```
char cities[5][30];
```

Memory Representation

char names[3][10] = {"Ravi","Anita","Kiran"};

Row 0	R	a	v	i	\0					
Row 1	A	n	i	t	a	\0				
Row 2	K	i	r	a	n	\0				

Every row has its own '\0'. The rows are stored contiguously.

Indexing

```
names[1]       // complete second string
names[1][0]    // first character of second string
names[2][3]    // fourth character of third string
```

First index

Selects the row / string.

Second index

Selects the character / column.

Example

names[2][3] → character 4 of row 2.

Initialization

```
char fruits[4][20] = {
    "Apple", "Banana", "Mango", "Orange"
};
```

- Each quoted string initializes one row.
- The compiler adds '\0'.
- The first dimension can be omitted: char fruits[][20] = {...};

Remember

Each row needs space for characters plus '\0'.

Partial Initialization

```
char names[5][20] = {
    "Ravi",
    "Anita"
};
```

- Only the first two rows are explicitly initialized.
- Remaining rows receive zero values.
- Useful when later input will fill the remaining rows.

Reading with scanf()

```
char names[5][20];

for (int i = 0; i < 5; i++)
    scanf("%19s", names[i]);
```

- names[i] is one complete string.
- The width 19 limits the input.
- %s stops at whitespace.
- Use this for single-word strings.

Reading Strings with Spaces

```
char names[3][40];

for (int i = 0; i < 3; i++) {
    fgets(names[i], sizeof(names[i]), stdin);
    names[i][strcspn(names[i], "\n")] = '\0';
}
```

Why fgets?

It accepts spaces, so full names such as “Ravi Kumar” can be read.

Newline

Remove the newline stored by fgets() before further processing.

Writing Multiple Strings

```
for (int i = 0; i < 4; i++)
    printf("%s\n", names[i]);

// or
puts(names[i]);
```

printf()

Use %s to print one string.

puts()

Prints a string followed by a newline.

Traversing Characters

```
for (int i = 0; i < rows; i++) {
    for (int j = 0; names[i][j] != '\0'; j++)
        printf("%c ", names[i][j]);
    printf("\n");
}
```

- Outer loop → string.
- Inner loop → characters.
- Stop at '\0' for each row.

Finding Length with strlen()

```
#include <string.h>

for (int i = 0; i < n; i++)
    printf("%s -> %zu\n", names[i],
strlen(names[i]));
```

strlen()

Counts characters before '\0'. It does not count '\0'.

Uses

Longest string, validation, statistics, formatting.

Accessing One String

```
char cities[4][20] = {
    "Hyderabad","Delhi","Pune","Chennai"
};

printf("%s", cities[2]);
```

cities[2] → "Pune"

A row behaves like a normal C string.

Accessing One Character

```
char names[3][20] = {"Ravi","Anita","Kiran"};

printf("%c\n", names[1][2]);
names[0][0] = 'r';
```

names[1][2]

Character index 2 of row 1 → 'i'.

Mutable

Characters in the array can be changed.

Using String Functions

```
strlen(names[i]);
strcpy(names[i], "NewName");
strcmp(names[i], "Ravi");
strcat(names[i], " Kumar");
```

- names[i] → one complete string.
- names[i][j] → one character.
- Include <string.h> for these functions.

Comparing Strings

```
for (int i = 0; i < n; i++) {
    if (strcmp(names[i], "Ravi") == 0)
        printf("Found Ravi\n");
}
```

strcmp() == 0

The strings contain the same characters.

Avoid == for strings

Use strcmp() to compare string contents.

Searching for a String

```
char key[20];
scanf("%19s", key);

for (int i = 0; i < n; i++) {
    if (strcmp(names[i], key) == 0) {
        printf("Found at %d\n", i);
        break;
    }
}
```

Linear search

Compare the key with each stored string.

Worst case

Every string may need to be checked.

Sorting Strings

```
for (int i = 0; i < n-1; i++)
    for (int j = i+1; j < n; j++)
        if (strcmp(names[i], names[j]) > 0) {
            char temp[20];
            strcpy(temp, names[i]);
            strcpy(names[i], names[j]);
            strcpy(names[j], temp);
        }
```

- strcmp() decides alphabetical order.
- strcpy() exchanges string contents.
- The same idea can be used for ascending or descending order.

Swapping Strings

```
char temp[20];

strcpy(temp, names[i]);
strcpy(names[i], names[j]);
strcpy(names[j], temp);
```

Why not = ?

Character arrays cannot be assigned with = after declaration.

Use strcpy()

Copy characters between arrays.

Finding Longest String

```
int maxIndex = 0;

for (int i = 1; i < n; i++)
    if (strlen(names[i]) > strlen(names[maxIndex]))
        maxIndex = i;

printf("Longest: %s", names[maxIndex]);
```

Logic

Store the index of the longest string found so far.

Output

Print names[maxIndex].

Counting Vowels

```
for (int i = 0; i < n; i++) {
    int count = 0;
    for (int j = 0; names[i][j] != '\0'; j++) {
        char ch = names[i][j];
        if (ch=='a'||ch=='e'||ch=='i'||ch=='o'||ch=='u')
            count++;
    }
    printf("%s -> %d\n", names[i], count);
}
```

String by string

↓

Character by character

↓

Apply condition

Counting Total Characters

```
int total = 0;

for (int i = 0; i < n; i++)
    total += strlen(names[i]);

printf("Total = %d", total);
```

Meaning

Total number of actual characters across all strings.

Extension

Average = (double)total / n.

Passing 2-D Strings to Functions

```
void display(char names[][20], int n) {
    for (int i = 0; i < n; i++)
        printf("%s\n", names[i]);
}

display(names, 5);
```

- The column size is normally specified in the parameter.
- Pass the number of valid strings separately.
- The function can process the original array.

Practical Example: Student Names

```
char names[5][30];

for (int i = 0; i < 5; i++) {
    printf("Enter name: ");
    scanf("%29s", names[i]);
}

for (int i = 0; i < 5; i++)
    printf("%d. %s\n", i+1, names[i]);
```

Flow

Declare → Input → Store → Traverse → Display

Limitation

%s reads one word. Use fgets() for full names.

Practical Example: Full Names

```
char names[3][40];

for (int i = 0; i < 3; i++) {
    fgets(names[i], sizeof(names[i]), stdin);
    names[i][strcspn(names[i], "\n")] = '\0';
}

for (int i = 0; i < 3; i++)
    puts(names[i]);
```

Example

Ravi Kumar
Anita Sharma
Kiran Reddy

Key point

fgets() accepts spaces.

Common Mistakes

Forgetting \0

Every string needs a terminator.

Wrong bounds

Use i < rows and j < columns.

Using == for strings

Use strcmp().

Unsafe input

Use width limits or fgets().

Check array size, input size, and '\0' carefully.

2-D Array vs Array of Pointers

```
char a[3][20] = {
    "Ravi", "Anita", "Kiran"
};

char *b[] = {
    "Ravi", "Anita", "Kiran"
};
```

2-D char array

Fixed storage per row; characters are stored in the array.

Array of pointers

Each element points to a string. String literals should not be modified.

Quick Review & Practice

```
char names[5][30];

for (int i = 0; i < 5; i++)
    scanf("%29s", names[i]);

for (int i = 0; i < 5; i++)
    printf("%s\n", names[i]);
```

- First index → string / row.
- Second index → character / column.
- Use string functions with names[i].
- Use nested loops for character processing.
- Practice: search, sort, longest name, vowel count.

Key takeaway: a 2-D char array is a practical way to store and process multiple C strings.